Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación
IIC2233 — Programación Avanzada 2026-1

Publicación: 9 de Abril de 2026
Actividad evaluada

Actividad 4
Excepciones, Iteradores e Iterables

Entrega

- **Lugar:** Repositorio personal de GitHub — Carpeta: `Actividades/AC4`
- **Fecha máxima de entrega:** 9 de Abril 17:20
- **Ejecución de actividad**: La Actividad será ejecutada **únicamente** desde la terminal del computador. **Los *paths* relativos utilizados en la Actividad deben ser coherentes con esta instrucción, y no pueden modificarse.**

Importante: Antes de comenzar, comprueba que Git este funcionando correctamente en tu repositorio privado. Para esto, **sube los archivos base de la actividad de inmediato** (*add*, *commit*, *push*). Se espera que en esta actividad (así como en las demás actividades y tareas) utilices Git a lo largo de **todo tu desarrollo** como una herramienta, no sólo como un método de entrega. Es por esto que recomendamos enfáticamente que vayas subiendo tus cambios constantemente (***push***), ya que **problemas de último minuto** relacionados con la entrega y Git **no serán considerados**.

Introducción

Fuiste contratado para programar un juego sobre resolver laberintos a ciegas. En este juego existe una grilla que representa el laberinto, y el jugador debe resolver el laberinto utilizando secuencias de movimientos.

Elementos del juego

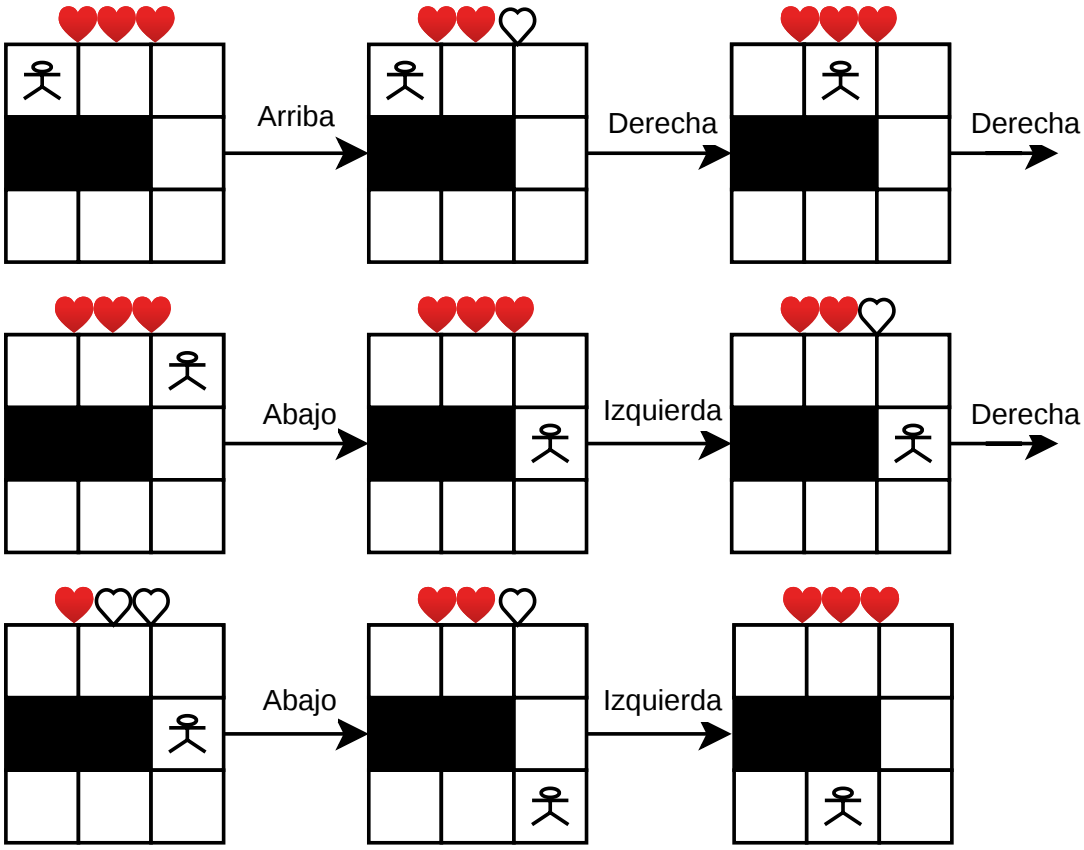
- El laberinto es una grilla rectangular donde los espacios pueden ser pasillo o muro. El personaje se puede mover libremente por los pasillos, pero no puede salir fuera de la grilla o pasar por los muros.
- El personaje tiene 3 vidas, y cada vez que intenta un movimiento prohibido, como moverse fuera del tablero o a un muro, pierde una vida. En cambio, si hace un movimiento permitido, recupera una vida hasta un máximo de 3.
- El jugador ingresa los movimientos como una secuencia de *strings*. Por ejemplo:

  ```
  ["ARRIBA", "ABAJO", "DERECHA", "DERECHA]
  ```

  Una vez ingresada la secuencia, el juego recrea los movimientos uno por uno, re-calculando su posición y vidas en cada movimiento.

En el siguiente ejemplo, el laberinto es el que se muestra en la imagen, con el jugador en la posición inicial $(0, 0)$. La secuencia de movimientos ingresada es:

```
["ARRIBA", "DERECHA", "DERECHA", "ABAJO",
 "IZQUIERDA", "ABAJO", "IZQUIERDA", "IZQUIERDA"]
```



En el ejemplo, la figura del personaje solo se mueve cuando el movimiento es válido. En cada movimiento inválido pierde una vida, y la recupera con cada movimiento válido, hasta un máximo de 3 vidas.

Archivos

El código base tiene 4 archivos:

- Modificar Entregar `tablero.py`: Contiene la `namedtuple Posicion` y la clase `Tablero`. Deberás modificarlo para resolver la Parte 1 de esta actividad.
- Modificar Entregar `camino.py`: Contiene las clases `CaminoIterable` y `CaminoIterador`. Deberás modificarlo para resolver la Parte 2 de esta actividad.
- Modificar `main.py`: Contiene código que puedes ejecutar para probar tu código. Modifícalo libremente si lo estimas conveniente.
- No modificar `utils.py`: Contiene el método `imprimir_tablero_con_posicion`. Es útil para visualizar el tablero. Úsalo en el main para probar el movimiento si lo estimas conveniente.

Parte 1. Movimientos en tablero

Tu objetivo es implementar la lógica de tablero y movimiento completando la clase `Tablero` en el archivo `tablero.py`. Se provee el siguiente código base:

- No modificar `Posicion`: `namedtuple` usada para representar posiciones en la grilla. Tiene los valores `fila` y `columna`.

- Modificar `Tablero`: Debes completar la clase `Tablero` para manejar el movimiento del personaje correctamente. Inicialmente tiene solos los métodos `__init__` y `mover`, pero puedes agregar más métodos si te es cómodo.

  - Modificar `def __init__(self, grilla: list[list[str]], inicio: Posicion):`

    - `self.grilla` Configuración del laberinto en forma de una grilla rectangular. Las celdas con el carácter `"."` representa un pasillo por el cual el personaje puede caminar, mientras que el carácter `"@"` representa un muro.
    - `self.inicio` Posición inicial del personaje, representada como una instancia de la `nametuple Posicion`. Por ejemplo: `Posicion(fila=0, columna=1)`. Puedes asumir que es una posición correcta dentro del laberinto que contiene pasillo.

  - Modificar `def mover(self, posicion: Posicion, direccion: str) -> Posicion:`

    Dada la posición del jugador y una dirección (`"DERECHA"`, `"IZQUIERDA"`, `"ABAJO"` o `"ARRIBA"`), retorna la nueva posición del jugador. En el caso de que el movimiento sea inválido, debe levantar una excepción. Específicamente debe manejar los siguientes casos:

    - Si la dirección ingresada no es uno de los *strings* de dirección (`"DERECHA"`, `"IZQUIERDA"`, `"ABAJO"` o `"ARRIBA"`), entonces levanta un `KeyError`.
    - Si la nueva posición del personaje está fuera del tablero, entonces levanta un `IndexError`.
    - Si la nueva posición del personaje es un muro, entonces levanta un `ValueError`.

    En otro caso, debe retornar la nueva posición del personaje usando la `namedtuple Posicion`.

Parte 2. Recorrer el camino con iterador

En el archivo `camino.py` existen las clases `CaminoIterable` y `CaminoIterador`. Estas permitirán recorrer una secuencia de direcciones y calcular las vidas del personaje mientras se mueve.

- No modificar `class CaminoIterable:` Clase iterable que contiene al Tablero y la lista de movimientos. Implementa la función `__iter__` que retorna un `CaminoIterador`. No debes modificar esta clase.
  - No modificar `def __init__(self, tablero: Tablero, movimientos: list[str]):`

    Inicializa los atributos `self.tablero` y `self.movimientos`
  - No modificar `def __iter__(self) -> CaminoIterator:`

    Retorna una instancia de la clase `CaminoIterador`.
- Modificar `class CaminoIterador:` Clase que maneja la iteración por sobre la secuencia de movimientos, la posición del personaje, y las vidas de este.
  - Modificar `def __init__(self, tablero: Tablero, movimientos: list[str]):`

    Inicializa los atributos `self.tablero` y `self.movimientos`. Agrega todo lo que necesites para manejar la secuencia de juego.

    En cada nueva iteración sobre un camino, el personaje debe partir con 3 vidas y en la posición inicial del tablero.
  - Modificar `def __next__(self) -> Posicion:`

    Itera sobre la lista de movimientos, moviendo al personaje en cada iteración y manejando las excepciones generadas por el tablero. También maneja el contador de vidas y la posición actual del personaje en cada iteración.

    Se deben programar los siguientes casos manejando las excepciones generadas por el tablero:

    1. Al inicio de cada iteración se debe imprimir `"Intentando movimiento <direccion>"`.

       Por ejemplo: `"Intentando movimiento ABAJO"`.
    2. Al intentar mover el personaje se deben manejar los casos:

       2.1 Si el personaje se intenta mover hacia un muro o fuera del tablero (`IndexError` o `ValueError`), debe perder una vida, no moverse y retornar la posición actual del personaje.

       2.2 Si el movimiento no es una dirección correcta (`KeyError`), se debe ignorar ese movimiento y no se pierden ni se ganan vidas. Luego, retornar la posición del personaje.

       2.3 Si el movimiento es correcto (no hay excepción), se debe mover al personaje a la nueva posición y sumar una vida hasta un máximo de 3 vidas. Luego retornar la posición del personaje.
    3. Luego de cualquier intento de movimiento se debe imprimir las vidas restantes con formato `"Vidas restantes <vidas>"`.

       Por ejemplo: `"Vidas restantes 3"`.

       En el caso de que se quede sin vidas debe se debe imprimir `"Has muerto"` luego de imprimir `"Vidas restantes 0"` y terminar de iterar sin retornar.

4. Si no quedan movimientos, se debe imprimir la posición final con formato `"Posición final (<fila>, <columna>)"`.

   Por ejemplo: `"Posición final (3, 2)"` y luego terminar de iterar.

Notas

- **No puedes hacer *import* de otras librerías externas a las entregadas en el archivo.**
- Tu actividad será evaluada usando tus archivos modificados `tablero.py` y `camino.py`. Los cambios en otros archivos no serán considerados en la corrección.
- Recuerda que la ubicación de tu entrega es en **tu repositorio de Git**. En la rama (*branch*) por defecto del repositorio: `main`.
- Se recomienda completar la actividad en el orden del enunciado.
- Recuerda que esta evaluación presenta corrección **automatizada**. Si entregas un código que se cae al momento de correr los *tests*, será evaluado con 0 puntos.
- Siéntete libre de agregar nuevos `print` en cualquier lugar para revisar objetos, modificaciones, etc... Es una herramienta muy útil para encontrar errores. Y si aparece un error inesperado, ¡léelo! Intenta interpretarlo.

Objetivos de la actividad

- Implementar clases siguiendo especificaciones técnicas.
- Levantar, y manejar excepciones en el código correctamente.
- Utilizar iteradores e iterables correctamente.
- Probar código mediante la ejecución de *test*.

Ejecución de *tests*

En esta actividad se provee de varios archivos `.py` los cuáles contiene diferentes *tests* que ayudan a validar el desarrollo de la actividad. Para ejecutar estos *tests*, **primero debes posicionar tu terminal/consola en la carpeta de la actividad (`Actividades/AC4`)**. Luego, desde esta misma, debes escribir el siguiente comando para ejecutar todos los *tests* de la actividad:

- `python3 -m unittest discover tests_publicos -v -b`

En cambio, para ejecutar los *tests* de cada parte de forma individual:

- `python3 -m unittest -v -b tests_publicos.tests_parte_1`
- `python3 -m unittest -v -b tests_publicos.tests_parte_2`

Finalmente, si deseas ejecutar un test en particular, puedes hacer usando el comando:

- `python3 -m unittest -v -b <carpeta>.<archivo>.<NombreClase>.<nombre_test>`

Por ejemplo:

- `python3 -m unittest -v -b tests_publicos.tests_parte_1.VerificarTablero.test_mover`

Importante: recuerda que si `python3` no funciona, probar con el comando específico de tu computador. Este puede ser `py`, `python`, `py3` o `python3.12`.